

Multiplicação de Matriz por Vetor (MxV): Acesso por Linhas vs Acesso por Colunas

Análise de Desempenho e Padrões de Acesso à Memória

Eugênio Vitor Lopes dos Santos

DCA3703 – Programação Paralela, Universidade Federal do Rio Grande do Norte

2026

1 Enunciado

Implemente duas versões da multiplicação de matriz por vetor (MxV) em C: uma com acesso à matriz por linhas (laço interno variando coluna) e outra por colunas (laço interno variando linha). Meça o tempo de execução de cada versão com uma função apropriada e execute testes com diferentes tamanhos de matriz. Identifique a partir de que tamanho os tempos passam a divergir significativamente e explique por que isso ocorre, relacionando suas observações com o uso da memória cache e o padrão de acesso à memória.

2 Fundamentação Teórica

2.1 Multiplicação de Matriz por Vetor (MxV)

A operação MxV calcula $y = A \cdot x$, onde A é uma matriz $N \times N$ e x é um vetor de tamanho N :

$$y_i = \sum_{j=0}^{N-1} A[i][j] \cdot x[j], \quad i = 0, 1, \dots, N-1$$

2.2 Ordem dos Laços e Disposição em Memória

Em C, matrizes bidimensionais são organizadas em ordem por linhas (*row-major order*). Elementos consecutivos da mesma linha ocupam posições contíguas na memória. A ordem de iteração nos laços determina diretamente se o acesso aproveita essa contiguidade física.

No acesso por linhas, o laço externo percorre i e o interno percorre j . A expressão $A[i \cdot N + j]$ visita $A[0][0]$, $A[0][1]$, $A[0][2]$, ... com passo unitário de 8 bytes (para o tipo `double`). Já no acesso por colunas, o laço externo percorre j e o interno percorre i . Cada iteração do laço interno avança N elementos na memória ($8N$ bytes), saltando linhas inteiras a cada operação.

2.3 Hierarquia de Cache e Localidade de Referência

A hierarquia de memória baseia-se em dois princípios fundamentais:

- **Localidade temporal.** Dados acessados recentemente permanecem no cache para acelerar acessos futuros imediatos.

- **Localidade espacial.** Ao buscar uma palavra da memória, o processador transfere um bloco contíguo inteiro (*cache line* de 64 bytes), antecipando acessos a endereços adjacentes.

O processador utilizado nos testes é um Intel Core i7-14700HX (20 núcleos físicos, 28 threads lógicas). A Tabela 1 descreve a topologia de cache da máquina.

Table 1: Hierarquia de cache da máquina de teste (Intel Core i7-14700HX).

Nível	Tipo	Capacidade
L1D	Dados, privativo por núcleo	48 KiB
L1I	Instruções, privativo por núcleo	32 KiB
L2	Unificado, privativo por núcleo	2 MiB
L3	Unificado, compartilhado	33 MiB

2.4 Linha de Cache e Transferência de Dados

A unidade atômica de transferência entre a memória principal e os níveis de cache é a linha de cache (*cache line*), de 64 bytes. Cada linha comporta exatamente 8 valores de tipo `double` (8 bytes cada).

No acesso por linhas, a primeira leitura traz 64 bytes contíguos da memória; os 7 acessos seguintes consomem os dados que já estão no cache L1, sem penalidade de latência adicional. No acesso por colunas com $N \geq 8$, cada leitura sucessiva busca um endereço deslocado por $8N$ bytes, caindo em uma linha de cache diferente. O processador carrega 64 bytes para utilizar apenas 8, desperdiçando 87,5% da largura de banda transferida.

3 Experimento

3.1 Código-Fonte

```

1 #define _POSIX_C_SOURCE 199309L
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <time.h>
5
6 static double agora(void) {
7     struct timespec t;
8     clock_gettime(CLOCK_MONOTONIC, &t);
9     return t.tv_sec + t.tv_nsec / 1e9;
10 }
11
12 int main(int argc, char *argv[]) {
13     int N = 2000;
14     if (argc > 1) N = atoi(argv[1]);
15
16     double *A = malloc((size_t)N * N * sizeof(double));
17     double *x = malloc(N * sizeof(double));

```

```

18     double *y = calloc(N, sizeof(double));
19
20     for (int i = 0; i < N * N; i++) A[i] = 1.0;
21     for (int i = 0; i < N; i++) x[i] = 1.0;
22
23     double inicio = agora();
24
25     for (int i = 0; i < N; i++) {
26         double s = 0.0;
27         for (int j = 0; j < N; j++)
28             s += A[i * N + j] * x[j];
29         y[i] = s;
30     }
31
32     double fim = agora();
33     double tempo = fim - inicio;
34     printf("Row-major N=%d tempo=%.6f s y[0]=%.1f\n",
35           N, tempo, y[0]);
36
37     free(A); free(x); free(y);
38     return 0;
39 }

```

Listing 1: Acesso por linhas (row-major)

```

1 #define _POSIX_C_SOURCE 199309L
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <time.h>
5
6 static double agora(void) {
7     struct timespec t;
8     clock_gettime(CLOCK_MONOTONIC, &t);
9     return t.tv_sec + t.tv_nsec / 1e9;
10 }
11
12 int main(int argc, char *argv[]) {
13     int N = 2000;
14     if (argc > 1) N = atoi(argv[1]);
15
16     double *A = malloc((size_t)N * N * sizeof(double));
17     double *x = malloc(N * sizeof(double));
18     double *y = calloc(N, sizeof(double));
19
20     for (int i = 0; i < N * N; i++) A[i] = 1.0;
21     for (int i = 0; i < N; i++) x[i] = 1.0;
22
23     double inicio = agora();
24
25     for (int j = 0; j < N; j++) {

```

```

26         for (int i = 0; i < N; i++)
27             y[i] += A[i * N + j] * x[j];
28     }
29
30     double fim = agora();
31     double tempo = fim - inicio;
32     printf("Column-major N=%d tempo=%.6f s y[0]=%.1f\n",
33           N, tempo, y[0]);
34
35     free(A); free(x); free(y);
36     return 0;
37 }

```

Listing 2: Acesso por colunas (column-major)

Os vetores e matrizes foram alocados e preenchidos antes de iniciar o cronômetro, isolando exclusivamente o laço de cálculo da multiplicação. O código foi compilado com GCC sem otimizações (-O0) para impedir que o compilador reordenasse os laços ou vetorizasse os acessos automaticamente:

```

1 gcc row_major.c -o row_major -O0
2 gcc column_major.c -o column_major -O0
3 ./row_major 2000
4 ./column_major 2000

```

Listing 3: Compilação e execução dos testes

4 Resultados

4.1 Dados Coletados

Table 2: Tempos de execução medidos para a multiplicação matriz-vetor.

N	Row-major (s)	Column-major (s)
10	0.002295	0.002831
20	0.001866	0.001827
30	0.001956	0.002252
40	0.002174	0.002068
50	0.002315	0.002776
100	0.001963	0.002202
200	0.002168	0.002965
300	0.002709	0.003243
400	0.003879	0.004282
500	0.005708	0.004723
600	0.005673	0.005695
700	0.006593	0.008838
800	0.008769	0.007560
900	0.010388	0.009706
1000	0.009775	0.011743
1200	0.013580	0.015531
1400	0.017189	0.018966
1600	0.019798	0.023868
1800	0.027032	0.034185
2000	0.031170	0.039495
2500	0.044809	0.069988
3000	0.071794	0.092323
3500	0.087319	0.140740
4000	0.115090	0.200937
4500	0.141870	0.239610
5000	0.154810	0.252379

4.2 Análise Visual

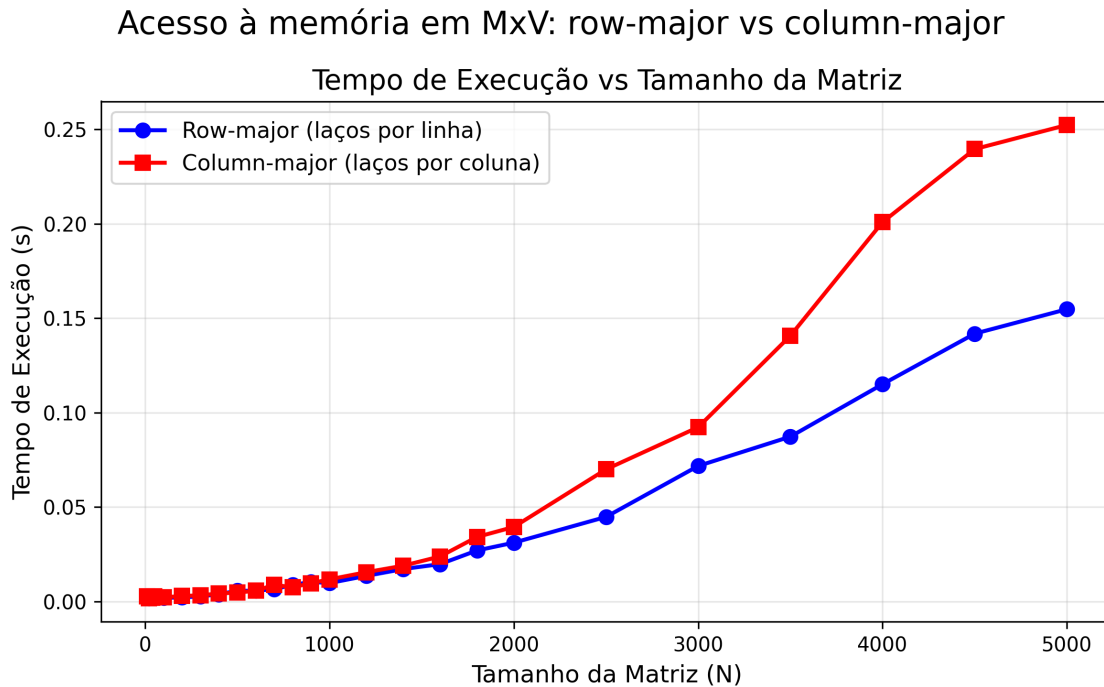


Figure 1: Tempo de execução em função do tamanho da matriz N . A divergência entre as versões acentua-se nos maiores tamanhos.

4.3 Comportamento Observado

Para matrizes pequenas ($N \leq 600$), os dados cabem nos caches L1 e L2, e ambas as versões completam em menos de 6 milissegundos. A variabilidade da medição supera a diferença real entre os padrões de acesso nessa faixa.

A separação começa a se manifestar na faixa intermediária ($700 \leq N \leq 1600$), embora de forma não monotônica: em $N = 500$, $N = 800$ e $N = 900$ a versão por colunas foi ligeiramente mais rápida, o que indica ruído experimental em medições de execução única. Esses casos não invalidam a tendência geral, mas reforçam a necessidade de repetições com mediana para isolar o sinal do ruído. Em $N = 1600$, com a matriz ocupando 20,48 MB e excedendo o cache L2 de 2 MiB, a versão por colunas levou 20,6% a mais (0,023868 s contra 0,019798 s).

Para matrizes grandes ($N \geq 2000$), a divergência consolida-se. Em $N = 2500$ a matriz consome 50 MB (8×2500^2 bytes), superando os 33 MiB do cache L3 compartilhado. As faltas de cache forçam buscas diretas na memória DRAM. Em $N = 5000$ (matriz de 200 MB), a versão por colunas levou 0,252379 s contra 0,154810 s da versão por linhas, um acréscimo de 63,0% no tempo total.

5 Discussão

5.1 Mecanismo de Divergência e Penalidade de Memória

A divergência decorre da perda de localidade espacial no acesso por colunas. Quando o laço interno varia a linha i com coluna j constante, iterações consecutivas leem $A[i \cdot N + j]$ e $A[(i + 1) \cdot N + j]$. A distância entre dois acessos sucessivos na memória é dada por:

$$\text{Stride} = N \times \text{sizeof}(\text{double}) = 8N \text{ bytes}$$

Para $N = 2000$, o salto entre iterações é de 16.000 bytes, equivalente a um deslocamento de 250 linhas de cache (16.000/64). Cada iteração consome apenas 8 bytes e descarta os outros 56 bytes trazidos pela linha de cache.

Enquanto a matriz cabe nos caches L1 (até $N \approx 78$) e L2 (até $N \approx 512$), a penalidade por falta de cache é baixa (4 a 14 ciclos de clock). Conforme o volume de dados excede o cache L2 e satura o cache L3 (33 MiB, superado em $N \approx 2080$), as faltas de cache recorrentes transferem a carga para a DRAM, cuja latência alcança 150 a 200 ciclos. O mecanismo de *hardware prefetcher* do processador também perde eficácia com passos de memória tão amplos, incapaz de antecipar o fluxo de dados não-contíguo.

5.2 Implicações em Computação de Alto Desempenho

O padrão de acesso à memória condiciona as técnicas de otimização em alto desempenho. Na vetorização SIMD, compiladores geram instruções vetoriais (AVX2, AVX-512) eficientes quando os operandos estão contíguos na memória; no acesso com stride, instruções do tipo *gather/scatter* degradam a taxa de transferência. Em aceleradores gráficos, threads vizinhas exigem acessos contíguos na memória global para viabilizar *memory coalescing*, e padrões com stride quebram essa fusão, elevando o tráfego no barramento. Bibliotecas de álgebra linear densa (BLAS) contornam essas limitações com blocagem (*tiling*) e transposição prévia de matrizes, garantindo que sub-blocos operem dentro do cache L1/L2 [1].

6 Conclusão

O experimento mostra que a ordem dos laços na multiplicação matriz-vetor determina o desempenho em arquiteturas modernas. Para matrizes pequenas ($N \leq 600$), os dados cabem nos caches L1 e L2, mantendo os tempos de ambas as abordagens equivalentes.

A divergência estabiliza-se a partir de $N \approx 1600$ e amplia-se quando o volume de dados supera os 33 MiB do cache L3 ($N \geq 2500$). Em $N = 5000$, o acesso por colunas leva 0,252 s contra 0,155 s do acesso por linhas, uma penalidade de 63,0% decorrente do desperdício de 87,5% de cada linha de cache e da sobrecarga na memória principal. Em C e em ambientes com armazenamento *row-major*, alinhar a iteração dos laços à disposição contígua da memória é necessário para o aproveitamento da hierarquia de cache.

References

- [1] J. L. Hennessy e D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6^a ed. Morgan Kaufmann, 2019.
- [2] W. A. Wulf e S. A. McKee, “Hitting the memory wall: Implications of the obvious,” *ACM SIGARCH Computer Architecture News*, vol. 23, n^o 1, pp. 20–24, 1995.
- [3] Intel Corporation, *Intel 64 and IA-32 Architectures Optimization Reference Manual*, 2024.